

OmniBrain Schema & Reusable Components

The new “MultimodalText” project will build on OmniBrain’s existing database and infrastructure, avoiding a rewrite of core systems. OmniBrain already logs **users, authentication, chat sessions, queries, agent executions, responses, citations, feedback, and metrics**. These tables can be reused for user accounts and conversation history. For example, ActiveWizards notes that a robust Text-to-SQL system “requires logging every question, the retrieved metadata, the generated SQL, and the final answer” for traceability [21†L118-L125]. OmniBrain’s tables like `queries`, `agent_executions`, `responses`, and `citations` can store exactly that information. We’ll also reuse `auth` and `auth_sessions` for login, and `feedback` for user ratings. In short, OmniBrain’s core tables (sessions, turns, queries, metrics, etc.) provide a ready-made audit trail, and can serve the new app with little change.

For example, production RAG/Text-to-SQL systems use such logs for observability and accuracy. ActiveWizards advises that “every time a user indicates a query was wrong... that pair (User Question, Correct SQL) should be saved” for future learning [21†L118-L125]. OmniBrain’s schema already supports storing these query-response pairs and feedback, so we can plug into existing logging and metrics without redesign.

MVP Architecture: Text-to-SQL + RAG

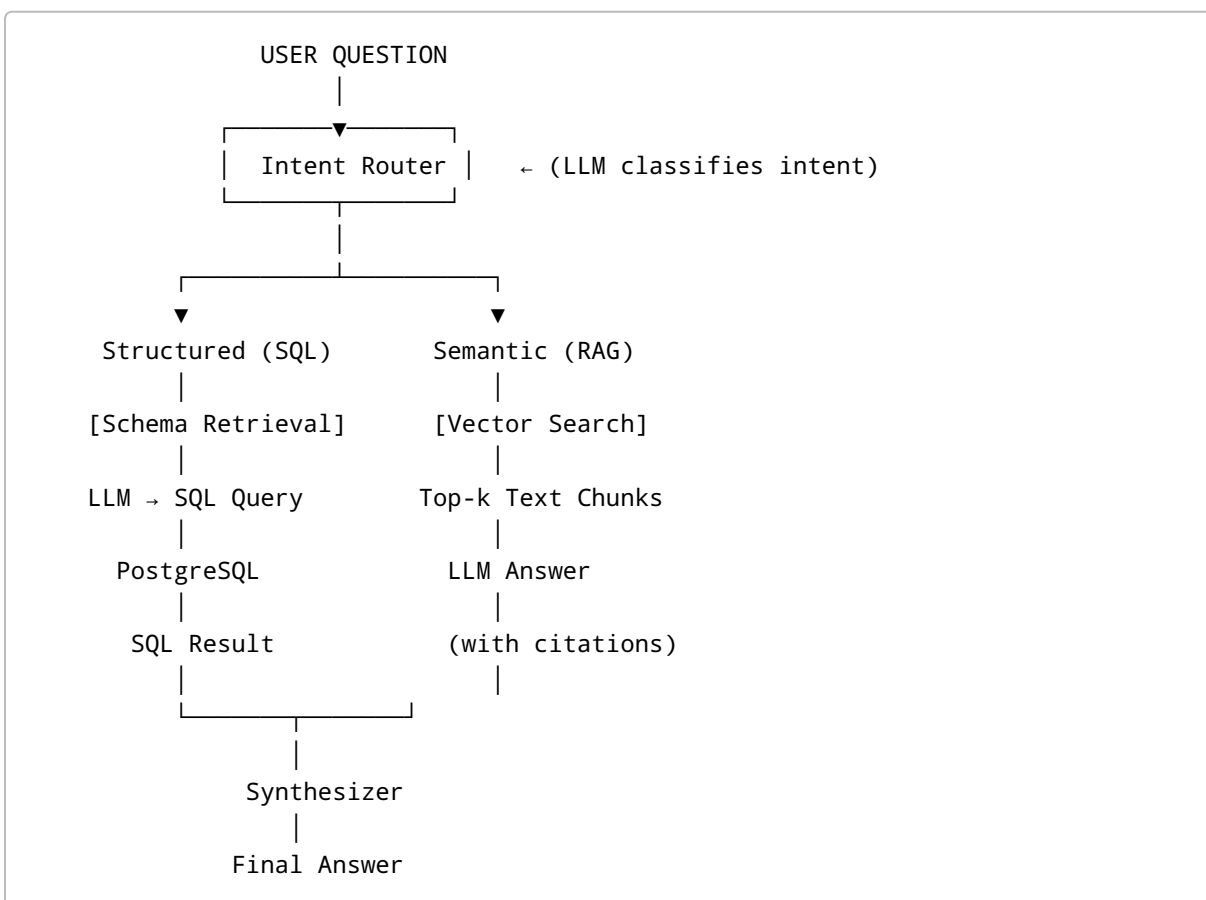
The core of the “MultimodalText” system will be a **hybrid pipeline** combining structured SQL querying with unstructured RAG-based retrieval, as shown below. An incoming user question is first classified by an *intent router* (an LLM or classifier) into two paths:

- **Structured Path (Text→SQL):** If the question pertains to tabular or structured data (e.g. “Show me total sales by region for Q1”), we route it through a Text-to-SQL agent. A retrieval component first *fetches relevant schema context* (table/column names, documentation, sample queries) from the database **schema**. Then an LLM generates the SQL query, which is validated and executed on PostgreSQL. For example, one design uses vector search over the database’s schema documentation to find relevant tables/columns before generation [18†L79-L87] [21†L78-L87]. This ensures the LLM has *context-aware* prompts (with table/column names and joins) to produce accurate SQL. The SQL is then executed (e.g. via `EXPLAIN` + `SQLAlchemy`), and the results can be post-processed (e.g. Pandas analysis or charts).
- **Semantic Path (RAG QA):** If the question is general or document-based, it flows into a RAG pipeline. We ingest all relevant text documents (manuals, reports, etc.) into a vector database (e.g. via Haystack or LangChain). Each doc is chunked, embedded, and stored (Ingestion & Embedding layers) [33†L34-L42]. At query time, the question is embedded and a nearest-neighbor search retrieves top-N chunks (Retrieval layer). These chunks (plus any conversation history) are injected into the LLM prompt for a grounded answer. For example, we might use GPT-4o or Google Gemini to synthesize the answer, citing the source chunks [33†L48-L50].

These two paths merge in a final *agent* step. A high-level LLM can combine the outputs (SQL results and retrieved text) to produce a cohesive answer. For instance, one pattern (“Agentic RAG with query routing”) has an LLM that decides: “Should I query the vector store or generate SQL?” and then calls the appropriate tool 【33†L75-L83】. In practice, we implement both flows in our API and use a simple router to pick the correct path based on intent.

Overall, this architecture matches best practices: **dynamic schema retrieval** for SQL generation 【18†L79-L87】 【21†L78-L87】, plus a standard RAG stack. The **five layers** of a RAG+SQL system (ingest, embed, retrieve, augment, generate) will be explicitly implemented 【33†L34-L42】 【33†L48-L50】. For example, the RAG portion will chunk docs, embed with an open model (E5, MiniLM, or GPT embeddings 【37†L100-L101】), store vectors in Qdrant/Milvus/pgvector 【37†L102-L103】, and retrieve via ANN search 【33†L34-L42】. The SQL portion will use PostgreSQL with its vector extension (pgvector) or a knowledge graph as needed. Finally, the Generation layer will use a powerful LLM (GPT-4o, Claude, or Gemini) to produce natural-language answers 【33†L48-L50】 【38†L108-L110】.

All components will communicate via a lightweight backend (e.g. FastAPI). We’ll containerize with Docker for portability. A simplified diagram:



Tools & Technology Stack

- **Programming Language & Frameworks:** We'll use **Python** for rapid AI development. Frameworks like **FastAPI** (or Flask) will host the backend API. For RAG orchestration, we can leverage a library like **LangChain** or **Haystack**. LangChain provides a structured way to build LLM pipelines (models, embeddings, retrievers) [28†L132-L137] . Haystack is a mature RAG framework with modular components for ingestion, retrieval, and generation [39†L19-L27] . Both support multiple LLMs and vector DBs without changing code. Additionally, LangChain's ecosystem tools (LangSmith, LangGraph) help debug and manage agents [28†L139-L147] [30†L800-L804] .
- **Vector Database:** For semantic retrieval, we'll use a vector store. Options include **pgvector** (Postgres extension), **Qdrant**, **Weaviate**, or **Milvus** [37†L102-L103] . Any of these can store embeddings of text or even images. For a quick MVP, pgvector is easy (embed vectors stored alongside documents in Postgres) [37†L102-L103] . We'll likely start with pgvector or Qdrant for simplicity. (Managed alternatives like Pinecone or Weaviate Cloud are also possible in future.)
- **Embeddings & LLM:** We'll use large LLM APIs such as **OpenAI's GPT-4o/GPT-4-Turbo**, **Google Gemini**, or **Anthropic Claude** for generation. For embeddings, we can use open models (e.g. Meta's Llama-3 embedders, Cohere's Embed-v3, OpenAI's `text-embedding-3`) [37†L100-L101] . The choice depends on cost/accuracy. For the MVP, OpenAI's embedding and chat endpoints (GPT-4o) are fast to integrate. We'll also implement fallback providers (e.g. Anthropic) for reliability.
- **Database & Storage:** The structured data (used by Text-to-SQL) and our vector store will live in **PostgreSQL**. OmniBrain already uses PostgreSQL (or a similar SQL DB), so we'll reuse that instance. PostgreSQL will hold tables for user data, ingestion (raw docs), and can even serve vectors via **pgvector**. This avoids adding extra DBs. We'll write SQLAlchemy models to interface with these tables.
- **Analysis & Visualization:** Python libraries like **Pandas** and **NumPy** will process SQL query results. For charts (if needed), **Plotly** or **Matplotlib** can generate visualizations. (Alternatively, a front-end like Streamlit or Dash could allow interactive dashboards in Day 3.)
- **DevOps & Tools:** The project will use **Docker** to containerize services (API, DB, RAG pipeline). Version control with GitHub/GitLab is assumed. We'll also use workflow automation (GitHub Actions or similar) to run tests. Given the short timeline, we rely on managed services (LLM APIs, cloud Postgres) as much as possible.
- **Additional Libraries:** For RAG-specific tasks, we may use deepset's Haystack library, or LlamaIndex for flexible indexing of PDFs/TXT. Both have built-in support for text splitting, embeddings, and retrieval [20†L393-L402] [38†L104-L110] . LangChain has similar capabilities via document loaders and chain abstractions. We'll pick one based on familiarity; for instance, Haystack's modular pipeline is well-documented [39†L19-L27] , and LlamaIndex explicitly mentions multi-modal support (text, images) [28†L250-L257] if we later add image input.

Overall, the stack is **Python + PostgreSQL + Docker + LLM APIs + a RAG framework**. All are open-source or widely adopted, so development speed is high and deployment is straightforward. We will integrate existing

OmniBrain modules for authentication, conversation logging, and metrics (feedback). For example, OmniBrain's query history and cache layers can be extended to cache RAG results or SQL answers, saving on API calls [33†L75-L83] [21†L118-L125] .

Phase-wise Development Plan (3 Days)

We'll adopt a rapid Agile approach, treating each day as a mini-sprint with clear goals and deliverables. Because OmniBrain provides much of the backend, we can focus on core functionality.

• Day 1: Setup & Data Ingestion

Tasks:

- **Environment & DB:** Spin up the Python/DB environment (Docker). Reuse the OmniBrain Postgres instance schema. Define any new tables (e.g. for uploaded documents or query logs). Ensure authentication is configured (reuse OmniBrain's auth tables). [33†L34-L42] shows that ingestion is the first step in RAG pipelines, so we focus here first.
- **Document Ingestion Pipeline:** Implement code to **ingest knowledge sources**. This includes uploading text/CSV/PDF data, chunking it, and storing chunks. For structured data (CSV/Excel), prepare a pipeline to read into SQL tables. We'll use a library (Haystack or similar) to split and index documents. As we ingest, we'll compute embeddings and insert vectors into the vector DB (pgvector or Qdrant). This builds the RAG index.
- **Schema Indexing:** For Text-to-SQL, create an index of database schema. We might store table/column names and descriptions in a simple vector store or even a local text file. This allows quick lookup of relevant tables. (ActiveWizards calls this "*dynamic metadata retrieval*" [21†L78-L87] .) We'll also set up any ORM models and test a basic SQL query through the API.
Deliverables: A working ingestion tool that accepts documents/CSVs and populates the RAG index and SQL tables. The database should now contain sample data and vector entries. We verify with a test query that embeddings and text chunks are stored.

• Day 2: Core Pipeline Implementation

Tasks:

- **Intent Routing:** Implement a simple intent classifier (could be a keyword-based switch or a small LLM prompt) to decide if a query is "structured" vs "semantic". For example, detect if user mentions specific fields or asks for aggregates → SQL. Otherwise → RAG. This corresponds to the *Intent Router* in our diagram [33†L75-L83] .
- **RAG Query Path:** Build the RAG query flow. Given a user question, use the vector DB to retrieve top-N relevant chunks. Then call the chosen LLM with these chunks + conversation history. Ensure the LLM prompt is constructed properly (with citations if possible). Test with an example Q&A. Use

frameworks' built-in prompts or custom prompts. We'll cite architectures that use LLM + retrieved context [18†L79-L87] [33†L48-L50] .

- **Text-to-SQL Path:** Build the SQL agent. Given a structured query, fetch relevant schema info (table/column names) – for example via a simple embedding lookup or even hard-coded schema tables. Then prompt the LLM to generate SQL. Immediately run the generated SQL against PostgreSQL to get results. (Include a validation step: use `EXPLAIN` to check cost, as recommended [21†L98-L100] , and block any unsafe statements.) We'll implement quick self-check or allow the LLM one retry if it hallucinates (simple heuristic).
- **Integration:** For questions that could use both (e.g. "Give sales by region, and also explain why X dropped"), combine paths: get SQL results and RAG insights, and merge them with a final LLM step if needed.
- **Basic UI/API:** Expose endpoints for user queries. Optionally create a minimal chat UI (e.g. Streamlit or React) to demonstrate the functionality.
Deliverables: A working backend that, for a query, correctly routes and returns answers. We should test both scenarios: e.g. "Show total sales in 2025" (SQL path) and "What is our refund policy?" (RAG path). We log the queries and answers in the OmniBrain tables (`conversation_turns` , `queries` , etc.).

• Day 3: Testing, Refinement & Documentation

Tasks:

- **Validation & Error Handling:** Refine the system for correctness. Add the self-reflection loop or a test query (as in Agentic design) if time permits. Ensure all SQL results are safe (read-only mode, deny 'DROP/DELETE', as recommended [21†L90-L98]). Handle cases where no relevant vector results are found (fallback to LLM text-only answers).
- **Performance & Metrics:** Integrate logging and monitoring. We should track metrics like query latency, RAG vs SQL hits, and user feedback. Tools like LangSmith or a simple DB logging can help [30†L800-L804] [30†L808-L811] . For example, we'll log each retrieved chunk and the citation, enabling us to audit answers. Add simple unit tests for the main flows.
- **Polish UI/UX:** Improve the interface. If we built a chatbot UI, make sure it displays answers and citations clearly. Possibly add charts (e.g. bar charts for numeric results using Plotly) if relevant.
- **Documentation & PDF Report:** Prepare the final deliverables. Write up documentation of architecture and usage. Assemble this phase-wise plan and tech choices into a PDF (as requested), including key design diagrams (we can create a simple flowchart) and references.
Deliverables: A finished prototype ready to demonstrate, plus project documentation. The documentation (to be delivered as PDF) will include: system architecture, technology stack, phase-by-phase summary, and timelines. We will ensure it cites best practices (e.g. from ActiveWizards and architecture guides) to justify our design decisions [18†L79-L87] [21†L118-L125] .

Throughout all phases, we'll apply best practices from the literature: e.g. keep the RAG ingestion and retrieval "visibly distinct" (see RAG design guidance [37†L115-L123]), include fallback paths, and test thoroughly. By leveraging OmniBrain's existing schema and a proven RAG framework, we can focus on the unique multi-modal components and still deliver the MVP in **3 days**.

Estimated Timeline Summary:

- **Day 1:** Environment & DB set up; implement ingestion/embedding (✓).
- **Day 2:** Build query paths (RAG & SQL) and integrate LLM answers (✓).
- **Day 3:** Testing, add guards (validation, cost-check), UI polish, and finalize documentation (✓).

Each day's work culminates in a demoable feature (data indexed, Q&A working, refined UI), ensuring we have tangible outputs at each phase.
